

Testar o Entity Framework Core (EF Core) usando **xUnit** envolve escolher entre testes unitários com mocks/in-memory ou testes de integração reais. A

recomendação moderna da comunidade e da Microsoft é priorizar testes de integração que validam o comportamento real do banco de dados. [1, 2, 3, 4]

Abaixo estão as principais abordagens e boas práticas para estruturar seus testes.

1. Abordagens para Testes com EF Core

- **Testes com Banco Real (Testcontainers - Recomendado):** Utiliza containers Docker (como PostgreSQL, SQL Server) para rodar uma instância real do banco durante os testes de integração. Garante fidelidade total ao comportamento do SGBD. [1, 2]
 - **SQLite In-Memory:** Abre uma conexão em memória com o SQLite. É rápido e suporta recursos relacionais que o provedor InMemory padrão do EF Core não suporta. [1, 2]
 - **EF Core InMemory Provider:** Armazena dados em estruturas de dicionário na memória RAM. É rápido, mas **não recomendado** para validar consultas complexas ou comportamentos específicos de banco de dados (pode mascarar bugs de SQL). [1, 2]
-

2. Estrutura Básica com `IClassFixture`

Para evitar recriar a infraestrutura do banco de dados a cada método de teste, utilize o recurso de **Class Fixture** do xUnit: [1, 2]

csharp

```
public class DatabaseFixture : IDisposable
{
    public DbContextOptions<MyContext> Options { get; private set; }

    public DatabaseFixture()
    {
        // Configuração de exemplo usando SQLite in-memory
        var connection = new SqlConnection("DataSource=:memory:");
        connection.Open();

        Options = new DbContextOptionsBuilder<MyContext>()
```

```
.UseSqlite(connection)
    .Options;

using var context = new MyContext(Options);
context.Database.EnsureCreated();
// Adicionar dados de seed se necessário
```

Use o código com cuidado.

3. Principais Recomendações

- **Evite o Provedor InMemory para SQL complexo:** Ele não traduz LINQ para dialetos reais de banco de dados, o que pode fazer um teste passar na memória e falhar em produção. [1]
- **Isolamento de Dados:** Garanta que um teste não interfira no estado do outro, limpando tabelas ou usando transações que sofrem rollback automático ao final do teste. [1, 2]
- **Testcontainers:** Se precisar testar recursos avançados de um banco específico (como concorrência ou funções nativas), prefira subir o banco via [Testcontainers for .NET](#) implementando `IAsyncLifetime` no xUnit. [1]